Chapter 5: Advanced SQL

- Procedural features of T-SQL
- Accessing SQL from a Programming Language

Database Principle and Design

5.1 Procedural features of T-SQL

- T-SQL provides following procedural features:
  - Variables
  - Operators
  - Built in Functions
  - Output Statements
  - Error Handling
  - Control Statements
  - Cursors
  - User-defined functions
  - Triggers
  - Stored procedures
- Note: Although the SQL standard defines some syntax for programming, most database systems implement nonstandard versions of these syntax. For example, the procedural languages supported by Oracle (PL/SQL), Microsoft SQL Server (T-SQL), and PostgreSQL (PL/pgSQL) all differ from the standard syntax.

5.2 Variables

- Two Types of Variables
  - Local variable
    - User defined variables in stored procedures, functions and batches
    - Name prefix: @
    - Examples: @*dept_name, @x*
  - Global variable
    - Defined by DBMS; users can only read global variables but cannot modify them
    - Name prefix: @@
    - Examples:
      - @@SERVERNAME: local server name
      - @@ROWCOUNT: the number of rows affected in last operation

```
select @@SERVERNAME
select * from department
select @@ROWCOUNT
```

Results | Messages

	(No column name)
1	ZHONGHONG-PC

	dept_name	building	budget
1	Biology	Watson	90000.00
2	Comp. Sci.	Taylor	100000.00
3	Elec. Eng.	Taylor	85000.00
4	Finance	Painter	120000.00
5	History	Painter	50000.00
6	Music	Pack...	80000.00
7	Physics	Watson	70000.00

	(No column name)
1	7

Local Variables

- Definition of a local variable

  declare *@variable_name* data_type, … …

  - *data_type* can be a system or user-defined data type
  - Example:

    declare *@dept_name* **varchar**(20), *@num* **int**

- The scope of a variable is the stored procedure, function or batch that declares the variable.
- Use local variables in SQL statements

  select * from *student where dept_name = @dept_name*

Set a value for a local variable

- A value can be set for a variable using a **set** statement

  set *@variable* = expression | select_statement;

  - Example:

    set *@dept_name* = 'History'

    set *@num* = (**select count**(*) **from** *student*)

    The **select** statement should only return 1 row and 1 column

- Set a local variable in the column list of a **select** statement

  select *@dept_name=dept_name, @num=tot_cred*

  from *student* **where** *name=*'Zhang'

  If the **select** statement return multiple rows, the variable takes the attribute value of last row.

5.3 Operators

- Algebraic operators: +, -, *, /, %
- Comparison operators: =, <>, >, >=, <, <=
- Logical operators: NOT, AND, OR
- String connection operators: +

select *ID + ', ' + name* **as** 'ID & Name'
from *student;*

ID & Name
00128, Zhang
12345, Shankar
19991, Brandt
22222, Hong
23121, Chavez
44553, Peltier
45678, Levy

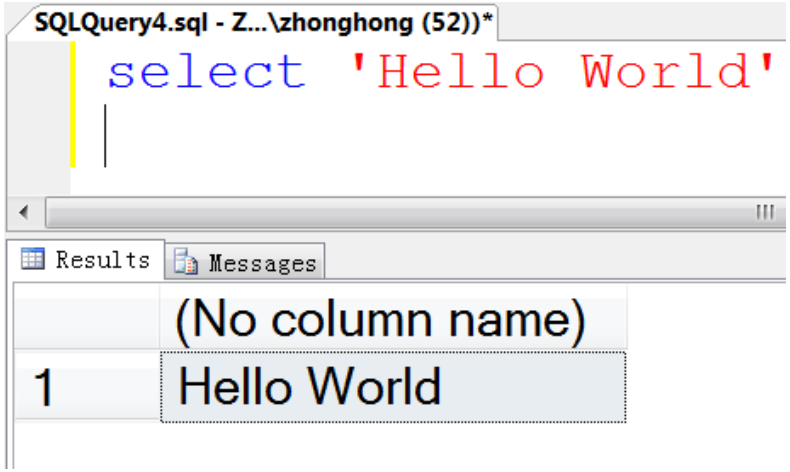
5.4 Built in Functions

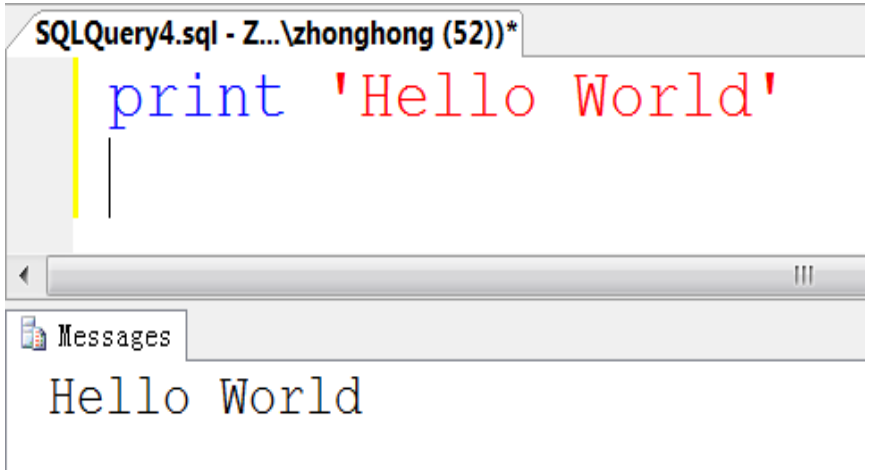
- Basic SQL functions (supported by standard SQL and T-SQL)
  - **AVG, SUM, MAX, MIN,COUNT**
- T-SQL functions (supported by T-SQL)
  - Data type transformation functions: **cast**, **convert**
  - String functions: **lower, upper, substring, …**
  - Algebraic functions: **abs, log, power, cos, sin, …**
  - Date and time functions: **getdate, year, month, day, datediff**

5.5 Output Statement

- **print** *string_expression*
  - Output general information to the client
  - Example:

    print 'Hello World'





5.6 Error Handling

Database Principle and Design

User-Defined Error

- **raiserror** (*string_expression, severity, state*)
  - Generate a user-defined error and initiates error processing for the session.
  - Example:

SQLQuery4.sql - Z...\zhonghong (52))*

```
print 'Before error'
raiserror ( 'You have a error', 13, 5)
print 'After error'
```

Messages

```
Before error
Msg 50000, Level 13, State 5, Line 2
You have a error
After error
```

Severity levels from 20 through 25 are considered fatal. Severity levels from 0 through 18 can be specified by any user; 19 - 25 can only be specified by members of the **sysadmin**

state Is an integer from 0 through 255. If the same user-defined error is raised at multiple locations, using a unique state number for each location can help find which section of code is raising the errors.

5.7 Control Statements

- **begin…end**
- **if…else**
- **while**
- **continue**
- **break**
- Example: Raise the prices of books by 1.1 times until the average price is greater than 500, and prevent the maximal price from being greater than 800

```
while  (select avg(price) from book) <= 500
        begin
                if (select max(price) from book) > 800
                        break;
                print 'Raise the prices!';
                update book set price = price * 1.1;
         end
```

5.8 Cursor

- **Cursor** is a tool that allows us to process the result of a **select** statement row by row.
- Declare a cursor

  declare *cursor_name* [**scroll**] **cursor**

  for *select_statement*

  - Example

    declare student_*cursor* **cursor**

    for select *ID, tot_cred* **from** *student*
- Open a cursor

  open student_*cursor*
- Close a cursor

  close student_*cursor*
- Deallocate a cursor

  deallocate student_*cursor*

Processing Data with Cursors

- Fetch a row and mark it as the current row

  fetch [**next** | **first** | **last** | **absolute** n]
  from *cursor_name* **into** *variable_list*

  - Example

    fetch next from student_cursor **into** @id, @cred

  - @@**fetch_status**: the status of the last fetch operation (0: successful; -1: failed)

  - Note: If **scroll** is not specified in an **declare cursor** statement, **next** is the only fetch option supported.

- Update current row

  update *student* **set** *name = 'alice'*

  where current of *student_cursor*

- Delete current row

  Delete from *student*

  where current of *student_cursor*

Example

- In EMS database, raise salary by different amount according the grade.

GRADE	INC(%)
1, 2	10
3, 4	8
>4	5

emp relation

empno	ename	job	mgr	hiredate	sal	comm	deptno
7369	SMITH	CLERK	7902	1980-...	880	NULL	20
7499	ALLEN	SALESMAN	7698	1981-...	1728	300	30
7521	WARD	SALESMAN	7698	1981-...	1375	500	30
7654	MARTIN	SALESMAN	7698	1981-...	1375	1400	30
7655	JONES	MANAGER	7839	1981-...	3213	NULL	20
7698	BLAKE	MANAGER	7839	1991-...	3078	NULL	30
7782	CLARK	MANAGER	7839	1981-...	2646	NULL	10
7788	SCOTT	ANALYST	7655	1987-...	3240	NULL	20
7839	KING	PRESIDENT	NULL	1981-...	5250	NULL	10
7844	TURNER	SALESMAN	7698	1981-...	1620	NULL	30
7876	ADAMS	CLERK	7788	1987-...	1210	NULL	20
7900	JAMES	CLERK	7698	1981-...	1045	NULL	30
7902	FORD	ANALYST	7655	1981-...	3240	NULL	20
7934	MILLER	CLERK	7782	1981-...	1430	NULL	10

salgrade relation

grade	losal	hisal
1	700	1200
2	1201	1400
3	1401	2000
4	2001	3000
5	3001	9999

```
DECLARE CUR SCROLL CURSOR
        FOR SELECT EMPNO,SAL FROM EMP;
OPEN CUR;
DECLARE @EMPNO INT, @SAL INT, @GRADE INT, @INC INT;
FETCH FIRST FROM CUR INTO @EMPNO, @SAL;
WHILE(@@FETCH_STATUS = 0)
BEGIN
        SELECT @GRADE=GRADE FROM SALGRADE
                WHERE @SAL BETWEEN LOSAL AND HISAL;
        IF @GRADE < 3 SET @INC = 10
        ELSE IF @GRADE < 5 SET @INC = 8
        ELSE SET @INC = 5;
        UPDATE EMP SET SAL=SAL+SAL*@INC/100
                WHERE CURRENT OF CUR;
        SELECT @EMPNO AS EMPNO, @SAL AS OLD,
                @SAL+@SAL*@INC/100 AS NEW; --FOR CHECKING
        FETCH NEXT FROM CUR INTO @EMPNO, @SAL;
END
CLOSE CUR;
DEALLOCATE CUR;
```

5.9 Stored Procedures

- A stored procedure is a set of SQL statements that can accomplish a certain task.
- It is stored in the database and invoked from SQL statements.
- It allow faster execution and can reduce network traffic
- It can be used as a security mechanism

Types of Stored Procedures

- System stored procedure
  - Provided by DBMS
  - With prefix “sp_” in names(eg. sp_spaceused)
  - Can be called in any database directly by their name.
- User stored procedure
  - Defined by users
  - Stored and executed in a database
  - Usually do not have a prefix in names

Create a Stored Procedure

- A stored procedure is created with a **create procedure** statement

  create procedure *procedure_name*
  @param1 data_type [**out**], *@param2* data_type [**out**], …
  as
  procedure_body

- The keywords **out** indicate parameters whose values are set in the procedure in order to return results.
- A **create procedure** statement should be the only statement in a batch.
- Example: given a department, find out the average salary of instructors in that department

  create procedure *avgsal*
  @dept varchar(20), @av numeric(8,2) **out**
  as
  select *@av=***AVG***(salary)* **from** *instructor*
  where *dept_name=@dept*
  go

- Execute a stored procedure

  declare *@a* **numeric**(8,2)
  execute *avgsal* 'Music', *@a* **out**
  select *@a*

- Drop a stored procedure

  drop procedure *avgsal*

Return

return [*integer_expression*]

- It quits the stored procedure, and return to where it is called
- *integer_expression*
  - An integer returned to the calling SQL statement
  - Usually it indicates the execution status of the procedure

```
CREATE PROCEDURE check_dept
        @name varchar(20), @dept varchar(20)
        AS
        IF(SELECT dept_name FROM instructor
                WHERE Name = @name) = @dept
                RETURN 1
        ELSE
                RETURN 0
GO
```

```
DECLARE @status int;
EXECUTE @status = check_dept 'Wu', 'Finance';
SELECT @status
```

5.10 User-Defined Functions

- Define a function that, given the name of a department, returns the number of instructors in that department.

```
create function dept_count (@dept_name varchar(20))
        returns int
begin
        declare @d_count  int;
                select @d_count =count (* ) from instructor
                where dept_name = @dept_name
        return @d_count;
end
```

- Functions can be used in **select** statements:

```
select dept_name, budget from department
where dbo.dept_count (dept_name ) > 2

select dept_name, dbo.dept_count (dept_name ) as num from department
```

Note:

- In SQL Server, a **create function** statement must be in a batch by its own
- A function must be invoked by using at least the two-part name of the function.
- If a function containing SQL queries is invoked on a large number of tuples, we may have serious performance problem.

5.11 Triggers

- A **trigger** is a procedure that is executed automatically by the system when the data in a table is modified.
- It can be called by the execution of an **insert**, **update**, or **delete** statement
- To design a trigger, we must:
  - Specify the conditions under which the trigger is to be executed.
  - Specify the actions to be taken when the trigger executes.
- Triggers introduced to SQL standard in SQL:1999, but supported even earlier using non-standard syntax by most databases.

Usage Scenario

- What is trigger used for?
  - Change related data automatically when a piece of data is changed
    - When giving a student a course grade, change the total credits of the student.
  - Enforce restrictions that are more complex than **check** constraints
    - Students can only retake a course if they fail in this course.
  - Evaluate the status of a table before or after data modification and take action accordingly.
    - If too many students take one course section, add a new section for this course.
  - Multiple triggers can be triggered for a single data modification action to achieve even complex effect.
    - When a new student is inserted, a user account is created and authorized

ID	course_id	sec_id	semester	year	grade
00128	CS-101	1	Fall	2017	A
00128	CS-347	1	Fall	2017	A-
12345	CS-101	1	Fall	2017	C
12345	CS-190	2	Spring	2017	A
12345	CS-315	1	Spring	2018	A

takes

student

ID	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56

Types of Triggers

- **after**
  - The trigger is fired only when all operations specified in the triggering SQL statement have executed successfully.
  - All referential cascade actions and constraint checks also must succeed before this trigger fires.
  - Multiple **after** triggers per **insert**, **update**, or **delete** statement can be defined on a table.
- **instead of**
  - The trigger is executed instead of the triggering SQL statement, therefore, overriding the actions of the triggering statements.
  - At most, one **instead of** trigger per **insert**, **update**, or **delete** statement can be defined on a table.
  - **instead of update/delete** triggers cannot be defined on a table that has a foreign key with a cascade on **update/delete** action defined.

Create a Trigger

create trigger *trigger_name* **on** *table_name*

{for| after | instead of} {[insert] [,] [update] [,] [delete]}

as

trigger_body

go

- Use **for** to specify an **after** trigger
- **create trigger** statement should be the only statement in a batch
- Drop a trigger

  drop trigger *trigger_name*

```
CREATE TRIGGER AfterTrigger ON
CLASSROOM
  FOR INSERT
AS
  PRINT 'In AfterTrigger'
  SELECT * FROM classroom
GO
```

```
CREATE TRIGGER InsteadOfTrigger ON
CLASSROOM
  INSTEAD OF INSERT
AS
  PRINT 'In InsteadOfTrigger'
  SELECT * FROM classroom
GO
```

How does a trigger work?

- Each trigger has two special logical tables
  - The **inserted** table
    - Contain the inserted rows of the trigger table
  - The **deleted** table
    - Contain the deleted rows of the trigger table
- An **update** operation is considered as first deleting the rows with old values and then inserting them back with new values
- The trigger can not modify those two lists, but can access them

```
CREATE TRIGGER TEST ON CLASSROOM
  FOR INSERT, UPDATE, DELETE
AS
  SELECT * FROM INSERTED
  SELECT * FROM DELETED
GO
```

Example 1

- Create a trigger that change the total credits of students when they are given course grades.

```
update takes set grade='B' where
ID=12345 and course_id='CS-315'
```

```
create trigger updategrade on takes
  for update
as
  update student set tot_cred =
    (select sum(credits) from takes, course
     where takes.course_id = course.course_id  and
           takes.ID = student.ID and
           takes.grade is not null and
           takes.grade <> 'F')
*  where ID in (select ID from inserted)
*go
```

Example 2

- Create a trigger to ensure that students can only retake a course if they fail in this course.

```
insert into takes values ('00128', 'CS-101',
'1', 'Spring', '2018', null);
```

```
create trigger retake on takes
  instead of insert
as
  if exists (select * from takes, inserted
             where takes.ID = inserted.ID and
                   takes.course_id = inserted.course_id and
                   (takes.grade is null or takes.grade <> 'F'))
     print 'students can only retake a course if they fail in
this course'
  else
     insert into takes select * from inserted
go
```

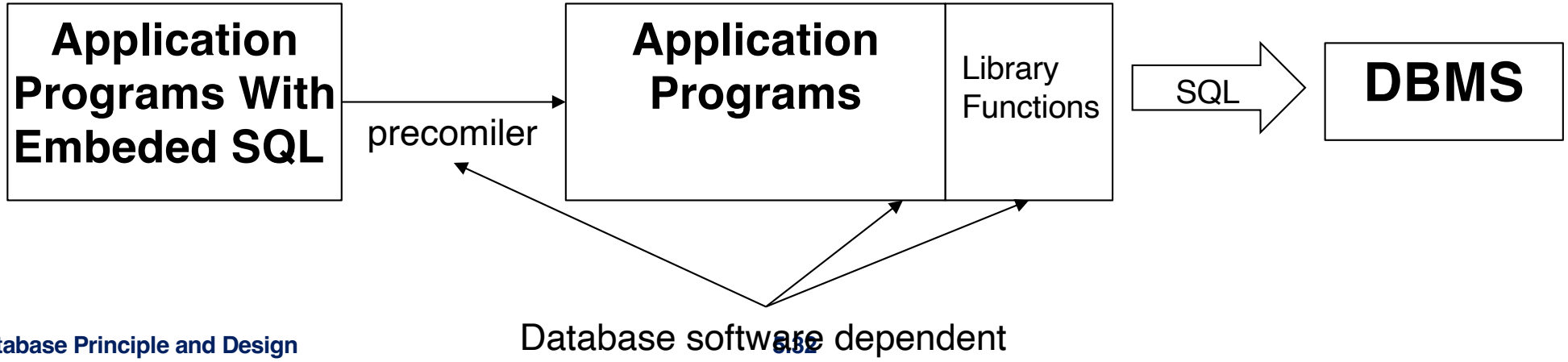
5.12 Accessing SQL from a Programming Language

- Most applications are written in a general-purpose programming language, such as C/C++, Java or Python.
- How to access SQL from the general programming language is very important for building applications that use database to manage data.

Embedded SQL

Embed SQL in application programs.

- A language in which SQL queries are embedded is referred to as a **host language**, and the SQL structures permitted in the host language constitute **embedded SQL** .
- The SQL standard defines embedding of SQL in a variety of programming languages, such as C, C++, Cobol, Pascal, Java, PL/I , and Fortran.
- Programs written in the host language can use the embedded SQL to access and update data stored in a database.
- An embedded SQL program must be processed by a special **preprocessor** prior to compilation. The preprocessor replaces embedded SQL with host-language declarations and function calls that allow runtime execution of the database accesses.



Example: Embeded SQL in C

```
int main() {
   EXEC SQL INCLUDE SQLCA;
   EXEC SQL BEGIN DECLARE SECTION;
      int OrderID;          /* Employee ID (from user)          */
      int CustID;             /* Retrieved customer ID          */
      char SalesPerson[10]    /* Retrieved salesperson name       */
      char Status[6]          /* Retrieved order status         */
   EXEC SQL END DECLARE SECTION;

   /* Set up error processing */
   EXEC SQL WHENEVER SQLERROR GOTO query_
   EXEC SQL WHENEVER NOT FOUND GOTO bad_number;

   /* Prompt the user for order number */
   printf ("Enter order number: ");
   scanf_s("%d", &OrderID);

   /* Execute the SQL query */
   EXEC SQL SELECT CustID, SalesPerson, Status
      FROM Orders
      WHERE OrderID = :OrderID
      INTO :CustID, :SalesPerson, :Status;

   /* Display the results */
   printf ("Customer number:  %d\n", CustID);
   printf ("Salesperson: %s\n", SalesPerson);
   printf ("Status: %s\n", Status);
   exit();

query_error:
   printf ("SQL error: %ld\n", sqlca->sqlcode);
   exit();

bad_number:
   printf ("Invalid order number.\n");
   exit();
}
```

Declare variables of the host language that can be used within embedded SQL statements.

Embeded **select** statement. Queries that return multiple rows of data are handled with cursor.

End of Chapter 5